



南開大學
Nankai University

南 开 大 学

软件工程课程作业：软件设计报告

DormFix 宿舍报修与维修进度管理系统

姓名： 王怡博，孙佳乐，何叶，王羿森

学号： 2310425，2310417，2313487，2310419

2026 年 6 月 3 日

目录

一、 引言	2
(一) 编写目的	2
(二) 项目背景	3
(三) 项目目标	3
(四) 术语与缩写说明	4
(五) 本章小结	4
二、 系统总体设计	4
(一) 系统功能概述	5
(二) 系统总体架构设计	7
(三) 系统整体结构图	8
(四) 技术选型说明	10
(五) 系统部署设计	11
(六) 本章小结	12
三、 详细设计	12
(一) 模块划分与模块功能设计	12
(二) 模块调用关系设计	15
(三) 核心业务流程设计	16
(四) 接口设计	18
(五) 本章小结	21
四、 UML 设计	21
(一) 用例图	21
(二) 类图	24
(三) 顺序图	27
(四) 工单状态图	30
(五) 部署图	31
(六) 本章小结	32
五、 数据库设计	32
(一) 数据库设计原则	32
(二) E-R 图	32
(三) 主要数据表设计	34
(四) 工单状态设计	36
(五) 本章小结	37
六、 系统质量设计	37
(一) 安全性设计	37
(二) 性能优化设计	39
(三) 异常处理机制	39
(四) 日志与监控设计	40
(五) 可扩展性设计	40
(六) 本章小结	41

七、 总结	41
(一) 系统设计成果	42
(二) 架构与设计特点	42
(三) 实际实现情况	42
(四) 后续优化方向	43
(五) 结语	43

一、 引言

本章阐明《DormFix 宿舍报修与维修进度管理系统软件设计报告》的编写目的、项目建设背景与目标，并统一全文术语。本章仅回答“为何建设”以及“本报告如何组织”，不涉及具体架构、模块与数据库等技术细节；后者自第二章起展开。详细业务需求以项目组已提交的《需求分析文档》为基线，本报告在其基础上给出可落地的设计方案与实现说明。

(一) 编写目的

1. 报告定位

本报告是 DormFix 项目在需求分析阶段之后形成的**软件设计说明文档**，用于将“系统应当做什么”转化为“系统如何组织与实现”。与需求分析文档侧重业务边界、用例与非功能指标不同，本报告重点描述系统总体架构、模块划分、核心流程、接口规范、UML 模型、数据库结构以及安全性与可扩展性等质量属性设计。

2. 编写作用

编写本设计报告旨在达成以下目标：

1. 为已基本完成的系统实现提供**可追溯的设计文档**，保证架构、数据模型与接口与代码一致，避免“实现与设计脱节”；
2. 在团队内部统一对分层架构、工单状态机、权限模型等关键设计的理解，降低维护与答辩演示阶段的沟通成本；
3. 为课程作业评审提供符合《软件工程课程作业：软件设计报告》要求的规范化交付物，涵盖总体设计、详细设计、UML、数据库及质量设计等内容；
4. 作为系统演示与后续优化（如消息推送、自动派单等）的**基线文档**，便于在不变更核心架构的前提下迭代扩展。

3. 与需求分析文档的关系

项目组已于需求分析阶段完成《宿舍报修与维修进度管理系统需求分析文档》，其中明确了用户角色、功能用例、业务流程、数据实体及非功能性需求。本报告**不重复**需求分析中的用例规约全文与风险分析章节，而在必要时引用其结论；若设计与需求存在差异（例如本期采用管理员人工派单而非需求文档中的自动派单设想），我们将在相应章节注明**设计取舍及原因**。

(二) 项目背景

1. 业务现状与痛点

国内多数高校的学生宿舍报修业务仍依赖人工登记、电话报修、微信群通知等分散渠道，信息化程度有限。结合本项目调研与需求分析结论，当前模式主要存在以下问题：

- **报修渠道分散且不规范**：报修信息格式不统一，关键字段（楼栋、房间、故障类别等）易缺失，难以形成可跟踪的标化工单；
- **信息传递效率低**：管理员需人工汇总后再通知维修人员，环节多、耗时长，紧急故障易因信息滞后而延误；
- **记录易遗漏与难以追溯**：纸质或聊天记录难以长期保存，纠纷发生时缺乏客观依据；
- **维修进度不透明**：学生提交报修后无法实时获知处理阶段，反复催办增加学生焦虑与管理人员负担；
- **统计与监管困难**：完成率、响应时长、维修人员工作量等指标难以量化，管理决策缺乏数据支撑；
- **缺乏评价与反馈闭环**：满意度与服务投诉难以系统收集，服务质量改进缺少依据。

2. 建设动因

上述痛点表明，传统报修模式已难以满足高校宿舍精细化管理与学生对住宿服务品质的合理期待。建设统一的线上报修与进度管理平台，有助于实现报修—审核—派单—维修—确认—评价的全流程数字化，并与智慧校园后勤信息化建设方向一致。

3. 系统简介

DormFix (Dormitory Fix) 为本项目英文名称，中文全称为 **宿舍报修与维修进度管理系统**。系统面向本校住宿学生、宿舍管理员及后勤维修人员，提供 Web 端统一入口，支持在线提交报修、工单审核与派单、维修进度跟踪、服务评价与投诉处理，以及面向管理员的数据统计与报表导出。系统采用 Django 与 Django REST Framework 构建后端，结合 Django Template 与 Tailwind CSS 实现 Web 界面，当前版本已在课程项目环境中完成主要功能开发与联调。

(三) 项目目标

1. 总体目标

建设一个**统一、高效、可追踪**的宿舍报修服务平台，通过信息化手段改善渠道分散、进度不透明、监管困难等现状，形成可审计的工单全生命周期管理能力。

2. 具体目标

结合需求分析文档与本期实现范围，系统应达成以下具体目标：

1. **统一报修入口**：学生通过 Web 端填写标准化报修表单（楼栋、房间、类别、描述、现场图片等），降低报修门槛；
2. **规范工单流转**：实现从待审核、待派单、已派单、处理中、待确认到已完成/已评价的完整状态机，并记录操作日志以支持时间线展示；
3. **提高协同效率**：管理员在线审核与派单，维修人员在线接单、更新状态与提交完工，减少电话与纸质传递；

4. **进度透明可查**: 学生与管理员可随时查看工单当前状态及历史操作记录, 缓解信息不对称;
5. **建立评价监督机制**: 支持多维度服务评分与投诉处理, 形成“报修—处理—反馈”闭环;
6. **支撑管理决策**: 为管理员提供工单量、完成率、响应/完成时长、满意度及维修人员绩效等统计视图, 并支持 Excel 报表导出。

(四) 术语与缩写说明

为便于阅读后续章节, 全文采用表1所列术语与缩写。表中释义与需求分析文档保持一致; 涉及实现细节的状态编码、接口路径等在第三至五章给出。

表 1: 术语与缩写说明

术语/缩写	释义
工单	一次完整的宿舍报修任务记录, 包含报修人、故障描述、地点、当前状态、处理结果及关联图片、日志等信息。
派单	宿舍管理员将审核通过的工单分配给指定维修人员的操作。
接单	维修人员确认接受已派发工单, 并将工单置为“处理中”等可作业状态的操作。
SLA	Service Level Agreement, 服务等级/时限协议; 本文泛指对响应或完成时间的管理目标, 本期以人工流程与状态跟踪为主。
用户	使用本系统的人员, 包括学生(student)、维修人员(maintainer)、管理员(admin)三类角色。
RBAC	Role-Based Access Control, 基于角色的访问控制; 本系统按角色限制页面与 API 访问范围。
DRF	Django REST Framework, 用于提供 REST 风格 API 的 Python 扩展框架。
Token 认证	用户登录后由服务端签发令牌, 客户端在后续请求头中携带 <code>Authorization: Token <token></code> 完成身份校验。
DormFix	本系统英文名称, 即宿舍报修与维修进度管理系统。

(五) 本章小结

本章明确了软件设计报告的编写目的、读者对象及其与需求分析文档的分工; 阐述了宿舍报修业务背景与 DormFix 建设动因; 给出了总体目标、具体目标及本期范围边界; 并通过术语表统一了全文关键概念。第三章将从宏观角度描述系统功能、总体架构与技术选型。

二、系统总体设计

本章从宏观层面描述 DormFix 系统的功能划分、总体架构、模块组织、技术选型及部署方案, 回答“系统如何组织与运行”。详细模块逻辑、业务流程与接口规范见第三章及后续章节。

(一) 系统功能概述

系统功能划分以需求分析文档中的三类用户角色为主线，按“学生端—维修人员端—管理员端—公共能力”组织。各角色功能与系统已实现页面对应关系见表2。

1. 学生端功能

学生用户为系统主要使用群体，核心功能包括：

- **提交报修**：填写楼栋、房间、故障类别、紧急程度、文字描述，并上传现场图片（系统限制最多 3 张、单张不超过 5 MB）；
- **查看工单**：浏览个人工单列表与详情，查看状态及操作时间线；
- **撤销工单**：对处于“待审核”状态的工单执行撤销；
- **确认维修**：维修完工后确认完成，或申请返修（工单回到处理中）；
- **服务评价**：对已完成的工单进行速度、态度、质量三维评分（1-5 分）及文字评价；
- **投诉反馈**：对维修服务提交投诉，并查看处理进度。

2. 维修人员端功能

维修人员通过 Web 工作台接收与处理任务，核心功能包括：

- **查看任务**：按状态筛选个人待办与历史任务，查看故障描述、地点及现场图片；
- **接收任务**：将“已派单”工单接单并置为“处理中”；
- **更新状态**：在处理过程中维护工单状态；
- **提交维修结果**：填写维修说明、耗材使用情况，将工单置为“已完成待确认”。

3. 管理员端功能

宿舍管理员负责审核、派单、用户与投诉管理以及数据分析，核心功能包括：

- **审核工单**：对待审核工单执行通过或驳回（驳回须填写理由）；
- **派发任务**：对待派单工单指定维修人员（受每日接单上限等业务规则约束）；
- **用户管理**：维护学生、维修人员、管理员账户，支持重置密码；
- **投诉处理**：查看并处理学生投诉，填写处理结果；
- **数据统计**：查看工单量、完成率、响应/完成时长、类别与状态分布、维修人员绩效及满意度，并导出 Excel 报表。

表 2: 用户角色与主要页面对应关系

角色	职责概要	代表性路径
学生	报修、查进度、确认、评价、投诉	/student/orders/, /student/repairs/create/
维修人员	接单、维修、完工提交	/maintainer/tasks/
管理员	审核、派单、用户、统计、投诉	/admin/pending-review/, /admin/dashboard/
通用	登录、个人中心	/login/, /profile/

4. 公共功能

三类角色共享的公共能力包括：统一登录（支持账号、学号/工号）、个人中心信息维护、基于 Token 的会话保持，以及按角色跳转至对应工作台。

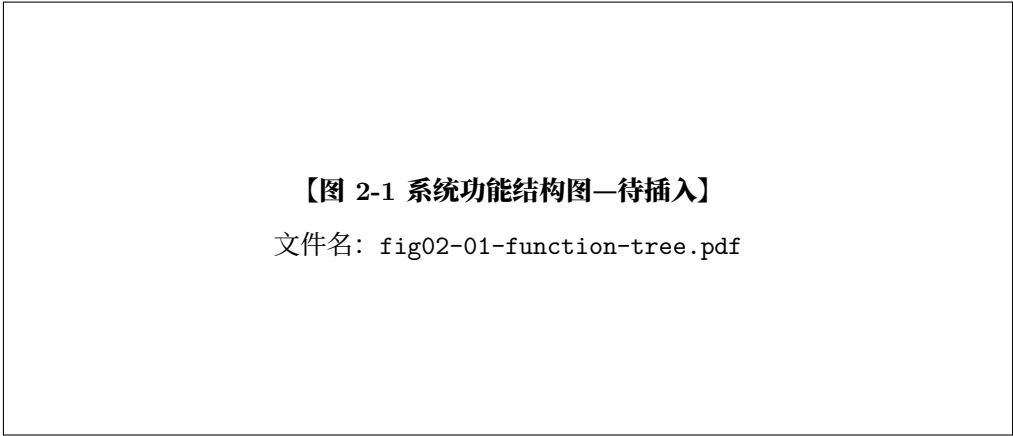


图 1: DormFix 系统功能结构图

图 2-1 制作说明（功能结构树图）：

1. **工具**：draw.io (diagrams.net)、Visio 或 StarUML 均可；
2. **根节点**：DormFix 宿舍报修与维修进度管理系统；
3. **一级子节点**（6 个）：用户管理；报修管理；派单管理；维修处理；评价投诉；数据统计；
4. **二级展开示例**：
 - 用户管理 → 登录认证、权限控制、用户维护、楼栋房间；
 - 报修管理 → 提交报修、工单查询、工单撤销；
 - 派单管理 → 工单审核、任务派发、维修人员列表；
 - 维修处理 → 接单、状态更新、完工提交、学生确认；
 - 评价投诉 → 服务评价、投诉申请、投诉处理；
 - 数据统计 → 工单统计、满意度分析、绩效统计、报表导出。
5. **版式**：自上而下树状或左向右组织架构图；框图统一宽高，连线为正交折线；
6. **导出**：PDF 矢量图，字号 ≥ 9pt，图中文字与正文术语一致；
7. **宽度**：LaTeX 中建议 width=0.82。

(二) 系统总体架构设计

1. 架构风格

DormFix 采用**浏览器/服务器 (B/S)** 模式：用户通过 Web 浏览器访问系统，无需安装专用客户端。服务端集中处理业务逻辑与数据持久化，便于统一部署与升级。

在 B/S 基础上，系统融合以下设计思想：

- **分层架构**：自顶向下划分为用户层、表示层、API 层、业务逻辑层、数据持久层，各层职责单一，降低耦合；
- **MVC 思想**：Django 框架原生体现 Model-View-Template 模式，Model 对应数据模型与 ORM，View 对应视图函数/类及 DRF 视图，Template 对应 HTML 页面；REST API 可视为面向资源的控制器扩展；
- **模块化业务划分**：按 Django App 拆分 accounts、repairs、dispatch、maintenance、feedback、dashboard 等应用，便于分工开发与维护。

选择该架构的原因包括：(1) 与课程项目技术栈及团队 Python 技能匹配；(2) 服务端渲染页面利于快速搭建管理端与学生端界面，同时 DRF 提供标准 API，形成“页面 + 接口”的半分离结构，兼顾演示效率与接口可测性；(3) 分层与 App 模块化便于在工单、派单等核心域独立演进。

2. 各层职责

表 3: 系统分层及各层职责

层次	职责说明
用户层	学生、管理员、维修人员通过浏览器访问各角色页面，发起报修、审核、派单、维修、评价等操作。
表示层	Django Template + Tailwind CSS + JavaScript；负责页面渲染、表单交互、Token 存储及通过 Fetch 调用后端 API。
API 层	Django REST Framework，统一前缀 <code>/api/</code> ，Token 认证、JSON 序列化、分页与权限校验。
业务逻辑层	各 App 的 views、serializers 及领域规则（状态机校验、派单上限、登录锁定等）。
数据持久层	Django ORM 访问 SQLite（开发）或 MySQL（部署），存储用户、宿舍、工单、评价、投诉等数据。

【图 2-2 系统总体架构图（逻辑分层）—待插入】

文件名: fig02-02-layer-architecture.pdf

图 2: 系统总体逻辑架构图

图 2-2 制作说明（逻辑分层架构图）：

1. **版式**：五层垂直堆叠矩形，自上而下为：用户层 → 表示层 → API 层 → 业务逻辑层 → 数据持久层；
2. **用户层**：框内画三个小框：学生端、管理端、维修端；
3. **表示层**：标注 “Django Template + Tailwind CSS + JavaScript”；
4. **API 层**：标注 “Django REST Framework (Token Auth)”；
5. **业务逻辑层**：横向排列六个模块名：accounts、repairs、dispatch、maintenance、feedback、dashboard（中文副标题：用户管理、报修管理、派单管理、维修处理、评价投诉、数据统计）；
6. **数据持久层**：标注 “SQLite / MySQL (Django ORM)”；
7. **连线**：层与层之间用单向箭头（向下表示请求，可旁注 “响应向上” 虚线箭头）；
8. **工具与导出**：draw.io 或 Visio；浅灰填充、黑色边框；导出 PDF 矢量图，宽度 0.78。

（三） 系统整体结构图

逻辑分层描述 “纵向职责”，本节从**模块组织**角度说明系统组成及协作关系。项目根包为 `dormfix`，业务按 Django App 划分；路由在 `dormfix/urls.py` 中集中注册，API 前缀为 `/api/<app>/`，页面路由按角色划分至 `/student/`、`/admin/`、`/maintainer/`。

表 4: Django 应用模块及职责

App 名称	核心模型	职责
accounts	User, DormBuilding, DormRoom	用户认证、登录锁定、个人信息、楼栋房间、管理员用户维护
repairs	WorkOrder, WorkOrderImage, WorkOrderLog	工单创建、查询、撤销、图片上传、状态日志
dispatch	(复用 repairs 模型)	工单审核、驳回、待派单列表、维修人员查询、派单
maintenance	(复用 repairs 模型)	维修人员任务列表、接单、完工、学生确认/返修
feedback	Evaluation, Complaint	三维评分、投诉提交与处理
dashboard	无独立业务表	统计聚合、图表数据、Excel 导出

模块间主要依赖关系为: **repairs** 提供工单核心实体; **dispatch**、**maintenance** 在工单状态变更时写入 **WorkOrderLog**; **feedback** 依赖已完成工单; **dashboard** 只读聚合各模块数据。**accounts** 为全局基础模块, 被所有业务 App 依赖。

【图 2-3 系统整体结构图 (模块/包) —待插入】

文件名: fig02-03-module-structure.pdf

图 3: 系统整体结构图 (前端、后端应用与数据库)

图 2-3 制作说明 (模块结构图):

1. **建议类型**: UML 组件图或部署无关的**逻辑结构图** (三列布局);
2. **左列—前端**: 一个大框 “Web 前端”, 内含: `templates/` (login、student、admin、maintainer)、`static/` (CSS/JS)、说明 “通过 Fetch 调用 API”;
3. **中列—后端**: 大框 “Django 应用 (dormfix)”, 内含 6 个小组件: accounts、repairs、dispatch、maintenance、feedback、dashboard; 每个组件下用注释列出 1-2 个关键 URL 前缀 (如 `/api/repairs/`);
4. **右列—数据库**: 框 “SQLite / MySQL”, 下列主要表名: user, dorm_building, dorm_room, work_order, evaluation, complaint 等;
5. **连线**: 前端 $\xrightarrow{\text{HTTP/JSON}}$ API 层 (或直连六个 App 的 API); 各 App $\xrightarrow{\text{ORM}}$ 数据库; dispatch、maintenance 用**虚线依赖**指向 repairs (复用模型);

6. **工具**：draw.io 组件图模板或 PlantUML component 图；

7. **导出**：PDF，宽度 0.88。

PlantUML 参考（可导出 PDF 后插入）：

```
@startuml
package "Web 前端" {
    [Templates + Tailwind + JS]
}
package "Django 后端" {
    [accounts] [repairs] [dispatch]
    [maintenance] [feedback] [dashboard]
}
database "SQLite/MySQL" {
    [业务表]
}
[Templates + Tailwind + JS] --> [accounts]
[Templates + Tailwind + JS] --> [repairs]
[dispatch] ..> [repairs]
[maintenance] ..> [repairs]
[accounts] --> [业务表]
[repairs] --> [业务表]
@enduml
```

（四） 技术选型说明

表 5: 技术选型及理由

类别	选用技术	选型理由
开发语言	Python 3	语法简洁、生态丰富，适合 Web 快速开发与课程项目周期。
Web 框架	Django 4.2	内置 ORM、认证、模板与管理后台，利于规范化的分层与 App 模块化。
API 框架	Django REST Framework 3.x	快速构建 RESTful API，内置序列化、权限、分页与 Token 认证。
数据库	SQLite（开发）/ MySQL（部署）	开发环境零配置；生产可切换 MySQL 以提升并发与可靠性。
前端	Django Template + Tailwind CSS	服务端渲染降低前后端分离成本；Tailwind 便于实现响应式界面。
脚本交互	JavaScript (Fetch API)	页面调用 /api/ 接口，实现登录态与动态数据加载。
图表	ECharts	管理端统计页折线图、饼图等可视化展示。
表格导出	openpyxl	将统计数据导出为 Excel，满足管理员报表需求。
图片处理	Pillow	校验与存储报修现场图片。
跨域	django-cors-headers	便于本地或前后端分离调试时配置 CORS。

认证方面，系统采用 DRF 的 **Token Authentication**：用户登录成功后颁发 Token，后续请求在 HTTP 头中携带 **Authorization: Token <token>**；配合基于角色的视图权限类，实现学生、维修人员、管理员的操作隔离。

（五） 系统部署设计

1. 部署环境

系统典型部署于校园内网或云服务器，用户通过浏览器访问。推荐软硬件环境如表6 所示。

表 6: 部署环境建议

项目	建议配置
客户端	Chrome / Edge / Firefox 等现代浏览器；支持桌面与移动端响应式访问。
Web 服务器	Nginx 反向代理静态资源与媒体文件，将动态请求转发至应用服务。
应用服务	Gunicorn + Django (WSGI)；开发阶段可使用 <code>manage.py runserver</code> 。
数据库	MySQL 5.7+ 或 8.0；开发阶段可使用 SQLite 文件数据库。
操作系统	Linux 或 Windows Server；课程演示可用 Windows/macOS 本机环境。

2. 部署拓扑

生产环境可采用“浏览器 → Nginx → Gunicorn/Django → MySQL”拓扑：Nginx 终止静态文件与上传图片访问；Django 进程处理业务与 API；MySQL 持久化业务数据。媒体文件（报修图片）存放于 MEDIA_ROOT，由 Nginx 或 Django 在 DEBUG 模式下按配置对外提供访问。



图 4: 系统物理部署图

图 2-4 制作说明（UML 部署图）：

1. **工具**：StarUML、draw.io（UML Deployment）或 PlantUML deployment；
2. **节点 1**：<<device>> 用户 PC / 手机—内部构件 “Web Browser”；
3. **节点 2**：<<device>> 应用服务器—构件 “Nginx”、“Gunicorn”、“Django App (dormfix)”；可用嵌套框表示；

4. **节点 3:** <<device>> 数据库服务器—构件 “MySQL”;
5. **通信:** Browser -(HTTP/HTTPS)-> Nginx -(WSGI)-> Django -(TCP)-> MySQL; 媒体文件可标注 Browser -(HTTP)-> Nginx 静态目录;
6. **课程演示简化版:** 可仅保留 Browser → Django Dev Server → SQLite 三节点, 图注中说明 “生产环境见虚线扩展”;
7. **导出:** PDF 矢量图, 宽度 0.75。
8. **与第 4 章关系:** 若第 4 章已绘部署图, 本章图 2-4 可与之保持一致或引用同文件。

(六) 本章小结

本章给出了 DormFix 的功能划分与角色职责, 阐述了 B/S、分层架构与 MVC 相结合的总体架构及各层职责, 说明了六个 Django App 的模块结构及依赖关系, 汇总了技术选型与部署拓扑。第三章将在模块与流程层面展开详细设计; 第四章、第五章将分别以 UML 图与数据表形式细化结构与数据设计。

三、 详细设计

本章在总体设计基础上, 对 DormFix 各功能模块的职责与业务规则、模块间协作关系、核心业务流程及 REST 接口进行细化说明。

(一) 模块划分与模块功能设计

系统按 Django App 划分为六个业务模块, 共享 `repairs` 中的工单实体。各模块通过 DRF 视图暴露 API, 由前端页面或管理脚本调用。表7 给出模块与核心模型的对应关系。

表 7: 模块划分概要

模块 (App)	核心模型	主要职责
accounts	User, DormBuilding, DormRoom	认证、权限、用户与楼栋房间维护
repairs	WorkOrder, WorkOrderImage, WorkOrderLog	报修创建、查询、撤销、日志记录
dispatch	复用 WorkOrder 等	审核、驳回、派单、维修人员负载查询
maintenance	复用 WorkOrder 等	接单、完工、学生确认/返修
feedback	Evaluation, Complaint	评价、投诉及投诉处理
dashboard	无独立业务表	统计聚合、图表数据、Excel 导出

1. 用户管理模块

模块标识: `accounts`。

功能范围:

- **登录认证:** 支持账号、学号/工号与密码登录; 成功返回 DRF Token;
- **登出:** 销毁服务端 Token;

- **个人信息**：已认证用户查看与修改姓名、电话等字段；
- **楼栋房间**：为报修表单提供宿舍楼、房间级联数据；
- **用户维护**：管理员对用户增删改查、重置密码（仅 admin 角色）。

核心业务规则：

1. 登录时优先按 account 查找用户，未命中则按 student_or_staff_no 查找；
2. 连续登录失败 5 次，账户锁定 15 分钟（lockout_until 字段）；
3. 密码使用 Django PBKDF2 哈希存储，最小长度 6 位；
4. 用户角色为 student、admin、maintainer 三者之一，后续 API 与页面按角色鉴权。

2. 报修管理模块

模块标识：repairs。

功能范围：

- 学生创建报修工单（房间、类别、描述、紧急程度、图片）；
- 按角色与条件查询工单列表（分页，默认每页 10 条）；
- 查看工单详情（含图片 URL、操作日志时间线）；
- 学生在“待审核”状态下撤销工单。

核心业务规则：

1. 仅学生可创建与撤销工单；学生仅可查看本人工单；
2. 未完成工单(状态为 pending_review、pending_dispatch、assigned、in_progress、pending_confirm) 合计达到 3 单时，禁止新建报修；
3. 仅 pending_review 状态允许撤销，撤销后状态为 cancelled；
4. 工单编号在首次保存时自动生成：WX + Unix 时间戳 + 4 位随机数；
5. 图片最多 3 张，单张不超过 5 MB，存储为 WorkOrderImage 记录；
6. 每次创建、撤销均写入 WorkOrderLog，便于前台时间线展示。

报修类别包括：水电、门窗、网络、家具、其他；紧急程度分为普通与紧急。

3. 派单管理模块

模块标识：dispatch（逻辑层独立，数据层复用 repairs）。

功能范围：

- 待审核工单列表、审核通过、审核驳回；
- 待派单工单列表、维修人员列表（含当日已派任务数）；
- 将工单派发给指定维修人员；
- 管理员查看全部工单（支持状态、类别筛选）。

核心业务规则：

1. 仅管理员可调用本模块接口；
2. 审核通过：pending_review → pending_dispatch；
3. 驳回须填写 reason，状态变为 rejected，理由写入日志备注；
4. 派单仅针对 pending_dispatch 工单，派单后状态为 assigned，记录 maintainer 与 assign_time；
5. 每位维修人员当日已派工单数 ≥ 5 时，拒绝继续派单。

4. 维修处理模块

模块标识： maintenance。

功能范围：

- 维修人员查看本人任务列表（默认筛选进行中相关状态）；
- 接收任务、提交维修完成；
- 学生确认完成或申请返修。

核心业务规则：

1. 接单：assigned → in_progress，仅指派给本人的工单可操作；
2. 完工：须填写 result（维修结果），可选填 materials（耗材），状态变为 pending_confirm，记录 finish_time；
3. 学生确认：confirmed=true 时 pending_confirm → completed；
4. 学生返修：confirmed=false 时回到 in_progress，须填写返修原因；
5. 自动确认：若工单处于 pending_confirm 且距 finish_time 已超过 3 天，学生在访问确认接口时系统将先自动置为 completed（日志记为“系统自动确认”）。

5. 评价投诉模块

模块标识： feedback。

功能范围：

- 学生对已完成工单提交三维评分（速度、态度、质量，各 1-5 分）及文字内容；
- 学生提交投诉；
- 管理员查看投诉列表并处理。

核心业务规则：

1. 评价仅允许 completed 状态且未存在评价记录的工单；
2. 评价成功后工单状态变为 evaluated，与工单一对一（Evaluation）；
3. 综合评分可在展示层按 $(speed + attitude + quality)/3$ 计算；
4. 评价文字经敏感词过滤（如含违规词则拒绝提交）；
5. 投诉关联工单与学生，管理员处理时更新状态与 process_result。

6. 数据统计模块

模块标识：dashboard。

功能范围：

- 汇总工单总数、完成率、平均响应时长与平均完成时长；
- 报修类别分布、工单状态分布、近 7 日每日报修趋势；
- 维修人员绩效排行（完工量、均工时、均评分、投诉量）；
- 满意度分布；Excel 报表导出（openpyxl）。

设计说明：本模块不定义独立 ORM 模型，通过 ORM 聚合查询 WorkOrder、Evaluation、Complaint 等表生成统计 JSON，供管理端 ECharts 图表与导出接口使用。仅管理员可访问。

（二） 模块调用关系设计

模块间依赖遵循“基础服务 → 核心实体 → 业务流程 → 反馈与统计”的顺序，如图5 所示。

- **accounts** 为全局依赖：所有模块通过 User 识别操作者，报修依赖 DormRoom 定位故障地点；
- **repairs** 为领域核心：维护工单主数据与日志；
- **dispatch**、**maintenance** 仅调用 **repairs** 模型及序列化器，不重复建表；
- **feedback** 依赖已完成工单，写入评价/投诉后回写工单状态；
- **dashboard** 只读依赖各业务表，不参与状态变更。

典型调用链示例：**学生报修**时，前端 → **accounts**（登录态）→ **repairs**（创建工单）；**管理员派单**时，**dispatch** → **repairs**.WorkOrder；**维修完工**后 **maintenance** → **repairs**，学生评价时 **feedback** → **repairs**。

【图 3-1 模块调用关系图—待插入】

fig03-01-module-call.pdf

图 5: 系统模块调用关系图

图 3-1 制作说明：自上而下绘制六模块方框，用实线箭头表示依赖：**dispatch**、**maintenance** → **repairs**；**feedback** → **repairs**；**repairs** → **accounts**；**dashboard** 以虚线箭头指向 **repairs**、**feedback**、**accounts**（标注“只读聚合”）。可用 draw.io 流程图或 PlantUML 组件依赖图导出 PDF。

(三) 核心业务流程设计

本节给出三条核心业务流程的活动说明及时序关系。对应活动图见图6；交互细节在第四章以顺序图补充。

1. 学生报修流程

参与角色：学生、系统（repairs 模块）、数据库。

流程步骤：

1. 学生登录系统，获取 Token；
2. 进入报修页面，选择楼栋、房间、类别，填写描述并可上传图片；
3. 系统校验：是否学生角色、未完成工单是否 < 3、必填项与图片规则；
4. 校验通过则创建 WorkOrder（初始状态 pending_review），保存图片并写入“提交报修”日志；
5. 返回工单详情/编号，学生可在工单列表查看进度；
6. 若工单仍为 pending_review，学生可选择撤销 → cancelled。

【图 3-2 学生报修业务流程图—待插入】

fig03-02-flow-repair.pdf

图 6: 学生报修业务流程图

图 3-2 制作说明（活动图/流程图）：使用 draw.io “流程图”或 StarUML Activity Diagram。符号：圆角矩形 = 步骤，菱形 = 判断（“是否学生”“未完成工单 < 3”“校验通过”），箭头标注“是/否”；否分支接“返回错误提示”并结束。主路径：登录 → 填写报修 → 校验 → 创建工单 → 写日志 → 结束。侧支：列表中“撤销”判断状态为待审核 → 已撤销。

2. 管理员审核与派单流程

参与角色：管理员、系统（dispatch）、维修人员（间接）。

流程步骤：

1. 管理员查看待审核列表（pending_review）；
2. **审核分支：**
 - 通过：状态改为 pending_dispatch，进入派单队列；

- 驳回：须填写理由，状态改为 `rejected`，流程终止。
3. 管理员查看待派单列表，查询维修人员及当日任务数；
 4. 选择维修人员执行派单，系统校验工单状态与人员当日负载 (< 5 单)；
 5. 派单成功：写入 `maintainer`、`assign_time`，状态 `assigned`，记录派单日志。

【图 3-3 审核与派单业务流程图—待插入】

fig03-03-flow-dispatch.pdf

图 7: 管理员审核与派单业务流程图

图 3-3 制作说明：分两段泳道（管理员 | 系统）。第一段：取待审核列表 → 判断“通过/驳回” → 更新状态。第二段：取待派单 → 选择维修员 → 判断“当日任务 < 5 ” → 派单成功/失败提示。

3. 维修处理流程

参与角色：维修人员、学生、系统（`maintenance`）。

流程步骤：

1. 维修人员在任务列表查看 `assigned` 等工单，执行接单 → `in_progress`；
2. 现场维修后提交完工，填写结果与耗材 → `pending_confirm`；
3. 学生查看工单，选择确认或返修：
 - 确认：→ `completed`，可进入评价；
 - 返修：→ `in_progress`，维修人员继续处理。
4. 若学生超过 3 天未操作，系统在确认接口触发自动确认 → `completed`；
5. 学生对 `completed` 工单评价 → `evaluated`（由 `feedback` 模块完成，可画在流程图末尾虚线框）。

【图 3-4 维修处理业务流程图—待插入】

fig03-04-flow-maintenance.pdf

图 8: 维修处理与学生确认业务流程图

图 3-4 制作说明：建议三泳道：维修人员 | 学生 | 系统。维修侧：接单 → 维修 → 提交完工。学生侧：确认/返修判断；系统侧：超时 3 天自动确认（可用定时判断菱形框）。可选后续框：提交评价 → 已评价。

（四） 接口设计

1. 接口设计原则

系统 REST API 遵循以下约定：

- **风格：**资源化 URL + HTTP 动词（GET 查询、POST 动作、PUT 更新）；
- **基础路径：** /api/<app>/，与 dormfix/urls.py 注册一致；
- **数据格式：**请求/响应主体为 JSON；文件上传采用 multipart/form-data；
- **认证：**除登录外，请求头携带 Authorization: Token <token>;
- **权限：**视图内校验角色，失败返回 HTTP 403；
- **分页：**列表接口支持 page、page_size（默认 10）；
- **错误：**参数错误 400，未认证 401，无权限 403，资源不存在 404；响应体含 error 或字段级错误信息。

2. 用户管理接口

表 8: 用户管理相关接口

方法	路径	权限	说明
POST	/api/accounts/login/	公开	请 求 体: <code>account</code> , <code>password</code> ; 成 功 返 回 <code>token</code> 与用户信息
POST	/api/accounts/logout/	已认证	销毁 <code>Token</code>
GET/PUT	/api/accounts/profile/	已认证	查询/更新个人信息 (姓名、电话等)
GET	/api/accounts/buildings/	已认证	宿舍楼列表
GET	/api/accounts/rooms/	已认证	房 间 列 表; 查 询 参 数 <code>building_id</code> 筛选
GET/POST	/api/accounts/users/	管理员	用户列表/创建用户
GET/PUT	/api/accounts/users/{id}/	管理员	用户详情/更新
POST	/api/accounts/users/{id}/reset-password/	管理员	重置指定用户密码

3. 报修管理接口

表 9: 报修管理相关接口

方法	路径	权限	说明
GET	/api/repairs/	已认证	工单列表; 学生仅本人; 支持 <code>status</code> , <code>category</code> 筛选
POST	/api/repairs/	学生	创建报修; 字段 <code>room</code> , <code>category</code> , <code>description</code> , 可选 <code>urgency_level</code> , 文件字段 <code>images</code>
GET	/api/repairs/{id}/	已认证	工单详情 (含图片、日志); 学生仅本人
POST	/api/repairs/{id}/cancel/	学生	撤销待审核工单

4. 派单管理接口

表 10: 派单管理相关接口

方法	路径	权限	说明
GET	/api/dispatch/all/	管理员	全部工单, 可筛选
GET	/api/dispatch/pending-review/	管理员	待审核列表
POST	/api/dispatch/{id}/approve/	管理员	审核通过
POST	/api/dispatch/{id}/reject/	管理员	驳回; 请求体 reason 必填
GET	/api/dispatch/pending-dispatch/	管理员	待派单列表
GET	/api/dispatch/maintainers/	管理员	维 修 人 员 列 表, 含 today_task_count
POST	/api/dispatch/{id}/assign/	管理员	派 单; 请 求 体 maintainer_id

5. 维修处理接口

表 11: 维修处理相关接口

方法	路径	权限	说明
GET	/api/maintenance/tasks/	维修人员	我的任务; 可选 status 筛选
POST	/api/maintenance/{id}/accept/	维修人员	接 单; assigned → in_progress
POST	/api/maintenance/{id}/complete/	维修人员	完工; 请求体 result 必填, materials 可选
POST	/api/maintenance/{id}/confirm/	学生	确认/返修; confirmed 布尔, 返修时 reason

6. 评价投诉接口

表 12: 评价投诉相关接口

方法	路径	权限	说明
POST	/api/feedback/evaluate/{work_order_id}/	学生	提 交 评 价; speed_score , attitude_score , quality_score (1-5) , content 可选
POST	/api/feedback/complaint/{work_order_id}/	学生	提交投诉; 含类型、内容等字段
GET	/api/feedback/complaints/	管理员	投诉列表
POST	/api/feedback/complaints/{id}/process/	管理员	处理投诉; 更新处理结果与状态

7. 数据统计接口

表 13: 数据统计相关接口

方法	路径	权限	说明
GET	/api/dashboard/statistics/	管理员	返回汇总指标、分布数据、趋势与绩效列表（JSON）
GET	/api/dashboard/export/	管理员	导出 Excel 报表文件

(五) 本章小结

本章细化了六个功能模块的职责与业务规则，给出了模块调用关系及学生报修、审核派单、维修确认三条核心流程，并列出了 REST 接口清单。第四章将以 UML 用例图、类图与顺序图对上述结构作形式化描述；第五章将给出数据表与工单状态机的数据层设计。

四、UML 设计

本章采用 UML 对 DormFix 系统的功能需求、静态结构与动态行为进行形式化描述。图形与第三章文字设计、项目实现代码保持一致；用例图在需求分析用例图基础上按最终实现做了精简与对齐。

(一) 用例图

1. 图意说明

用例图描述三类参与者与系统功能用例之间的交互边界。参与者（Actor）包括：

- **学生**（Student）：报修发起人；
- **维修人员**（Maintainer）：执行维修任务；
- **宿舍管理员**（Admin）：审核、派单、用户与投诉管理、数据统计。

系统用例与需求分析文档编号（UC001-UC012）的对应关系见表14。图中对“包含（include）”关系的使用说明：多个用例依赖“用户登录”，故以 <<include>> 方式关联 UC001，避免重复连线。

表 14: 主要用例与需求编号对应

参与者	用例名称	编号	实现要点
全体	用户登录	UC001	Token 认证、登录锁定
学生	提交报修申请	UC002	最多 3 个未完成工单
学生	查看工单进度	UC003	时间线日志
学生	撤销未审核工单	UC004	仅 pending_review
管理员	审核报修申请	UC005	通过/驳回
管理员	派发维修任务	UC006	日派单上限 5
维修人员	接收维修任务	UC007	接单
维修人员	完成维修	UC008	待确认
学生	确认维修结果	UC009	确认/返修/自动确认
学生	提交服务评价	UC010	三维评分
学生	提交投诉	UC011	敏感场景投诉
管理员	处理投诉	UC012	更新处理结果
管理员	用户管理	—	增删改查、重置密码
管理员	数据统计与导出	—	图表与 Excel

【图 4-1 系统用例图—待插入】

fig04-01-usecase.pdf

图 9: DormFix 系统用例图

图 4-1 制作说明：

1. **工具**：StarUML、Visual Paradigm、draw.io (UML Use Case) 或 PlantUML;
2. **系统边界**：矩形框标题为 “DormFix 宿舍报修与维修进度管理系统”;
3. **左侧参与者**：学生（人形图标）;
4. **右侧参与者**：维修人员（上）、宿舍管理员（下）;
5. **学生关联用例**：提交报修、查看工单、撤销工单、确认维修、提交评价、提交投诉（均 <<include>> 登录）;

6. **维修人员关联**: 接收任务、完成维修、查看任务 (include 登录);
7. **管理员关联**: 审核报修、派发任务、处理投诉、用户管理、数据统计 (include 登录);
8. **泛化**: 无需为三类用户建“用户”父参与者 (保持与需求文档一致);
9. **导出**: PDF 矢量, 横向排版, width=0.95。

PlantUML 参考 (导出 PDF 后插入图 4-1):

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor 学生 as Student
actor 维修人员 as Maintainer
actor 宿舍管理员 as Admin

rectangle "DormFix系统" {
    usecase "用户登录" as UC1
    usecase "提交报修申请" as UC2
    usecase "查看工单进度" as UC3
    usecase "撤销未审核工单" as UC4
    usecase "确认维修结果" as UC9
    usecase "提交服务评价" as UC10
    usecase "提交投诉" as UC11
    usecase "接收维修任务" as UC7
    usecase "完成维修" as UC8
    usecase "审核报修申请" as UC5
    usecase "派发维修任务" as UC6
    usecase "处理投诉" as UC12
    usecase "用户管理" as UM
    usecase "数据统计与导出" as ST
}

Student --> UC2
Student --> UC3
Student --> UC4
Student --> UC9
Student --> UC10
Student --> UC11
Maintainer --> UC7
Maintainer --> UC8
Admin --> UC5
Admin --> UC6
Admin --> UC12
Admin --> UM
Admin --> ST
```



```

UC2 ..> UC1 : <<include>>
UC3 ..> UC1 : <<include>>
UC5 ..> UC1 : <<include>>
UC6 ..> UC1 : <<include>>
@enduml

```

(二) 类图

1. 图意说明

类图描述系统核心业务实体及其关联，对应 Django ORM 模型，主要关系如下：

- DormBuilding 与 DormRoom：一对多 (1..*)；
- User 与 WorkOrder：学生侧一对多；维修人员侧 0..*；
- DormRoom 与 WorkOrder：多对一；
- WorkOrder 与 WorkOrderImage、WorkOrderLog：一对多；
- WorkOrder 与 Evaluation：一对一；
- WorkOrder 与 Complaint：一对多；
- User 与 WorkOrderLog、Complaint：操作人/投诉人关联。

表 15: 核心业务类主要属性

类名	主要属性
User	account, name, studentOrStaffNo, phone, role, status, loginAttempts, lockoutUntil, createdAt
DormBuilding	buildingCode, buildingName, genderLimit
DormRoom	roomNo, floorNo
WorkOrder	orderNo, category, description, urgencyLevel, status, submitTime, assignTime, finishTime, cancelFlag
WorkOrderImage	image
WorkOrderLog	fromStatus, toStatus, operationType, operationTime, remark
Evaluation	speedScore, attitudeScore, qualityScore, content, createdAt
Complaint	type, content, status, processResult, createdAt, handledAt

【图 4-2 系统类图—待插入】

fig04-02-class.pdf

图 10: 系统核心业务类图

图 4-2 制作说明:

1. 每个类分三格: 类名 / 属性 / 方法 (方法可仅列 +avgScore() 等关键项);
2. 关联线上标注多重性: 如 Building “1” — “*” Room;
3. Evaluation 与 WorkOrder 用一对一实心菱形或 1..1 标注;
4. 不画 Django 框架内部类 (如 Model), 保持业务可读性;
5. 工具推荐 StarUML “Class Diagram” 或 PlantUML class 图;
6. 导出 PDF, width=0.92, 字号不小于 9pt。

PlantUML 参考:

```
@startuml
skinparam classAttributeIconSize 0
```

```
class User {
    +account: String
    +name: String
    +role: String
    +phone: String
    +loginAttempts: int
}
```

```
class DormBuilding {
    +buildingCode: String
    +buildingName: String
}
```

```
class DormRoom {  
    +roomNo: String  
    +floorNo: int  
}
```

```
class WorkOrder {  
    +orderNo: String  
    +category: String  
    +status: String  
    +description: String  
    +submitTime: DateTime  
}
```

```
class WorkOrderImage {  
    +image: String  
}
```

```
class WorkOrderLog {  
    +operationType: String  
    +fromStatus: String  
    +toStatus: String  
}
```

```
class Evaluation {  
    +speedScore: int  
    +attitudeScore: int  
    +qualityScore: int  
}
```

```
class Complaint {  
    +type: String  
    +status: String  
    +content: String  
}
```

```
DormBuilding "1" -- "*" DormRoom  
User "1" -- "*" WorkOrder : student >  
User "0..1" -- "*" WorkOrder : maintainer >  
DormRoom "1" -- "*" WorkOrder  
WorkOrder "1" -- "*" WorkOrderImage  
WorkOrder "1" -- "*" WorkOrderLog  
User "1" -- "*" WorkOrderLog : operator >  
WorkOrder "1" -- "0..1" Evaluation  
WorkOrder "1" -- "*" Complaint
```

```
User "1" -- "*" Complaint : student >
@enduml
```

(三) 顺序图

顺序图用于描述核心用例在运行时的对象交互与时序。以下给出两个课程重点用例的顺序图说明。

1. 提交报修顺序图

用例：UC002 提交报修申请。

参与对象：:Student (学生)、:WebPage (报修页面)、:RepairAPI (repairs 视图)、:WorkOrder (工单实体)、:Database (数据库)。

交互概要：

1. 学生填写表单并提交，页面携带 Token 调用 POST /api/repairs/;
2. API 校验角色、未完成工单数及必填项;
3. 创建 WorkOrder 与 WorkOrderImage，写入 WorkOrderLog;
4. 返回 201 及工单 JSON，页面提示成功并跳转列表。

【图 4-3 提交报修顺序图—待插入】

fig04-03-seq-repair.pdf

图 11: 提交报修申请顺序图 (UC002)

图 4-3 制作说明：生命线自上而下为：学生、WebPage、RepairAPI、WorkOrder、Database。消息顺序：submitForm() → POST /api/repairs/ → validate() → create() → INSERT → createLog() → 返回 orderJSON。在 validate() 处用 alt 片段：“校验失败”返回 400；“未完成工单 ≥ 3”返回 403。

PlantUML 参考：

```
@startuml
actor 学生 as Student
participant "报修页面" as Page
participant "RepairAPI" as API
participant "WorkOrder" as WO
```

```
database "数据库" as DB

Student -> Page : 填写并点击提交
Page -> API : POST /api/repairs/ (Token, formData)
activate API
API -> API : 校验角色与未完成工单数
alt 校验失败
    API --> Page : 400/403 错误
else 校验通过
    API -> WO : create(room, category, ...)
    WO -> DB : INSERT work_order
    API -> DB : INSERT images, log
    API --> Page : 201 + orderJSON
end
deactivate API
Page --> Student : 提示提交成功
@enduml
```

2. 派发维修任务顺序图

用例：UC006 派发维修任务（含审核通过后的派单阶段）。

参与对象：:Admin、:DispatchAPI (dispatch 视图)、:WorkOrder、:User（维修人员）、:Database。

交互概要：

1. 管理员在待派单列表选择工单与维修人员；
2. 调用 POST /api/dispatch/{id}/assign/, 请求体含 maintainer_id;
3. API 校验工单为 pending_dispatch, 统计维修员当日派单数;
4. 更新工单 maintainer、assign_time、status=assigned, 写日志;
5. 返回成功, 前端刷新; 维修人员可在任务列表看到新工单（图中可用注释说明）。

【图 4-4 派发维修任务顺序图—待插入】

fig04-04-seq-dispatch.pdf

图 12: 派发维修任务顺序图 (UC006)

图 4-4 制作说明：生命线：管理员、DispatchAPI、WorkOrder、User (Maintainer)、Database。关键alt：当日任务数 ≥ 5 时返回错误；否则更新工单并写日志。可在图末以注释框表示维修人员“被动接收”新任务（无需同步消息）。

PlantUML 参考：

```
@startuml
actor 管理员 as Admin
participant "DispatchAPI" as API
participant "WorkOrder" as WO
participant "维修人员" as M
database "数据库" as DB

Admin -> API : POST /dispatch/{id}/assign/\n(maintainer_id)
activate API
API -> WO : getById(id)
API -> DB : COUNT 今日派单数
alt 任务数 >= 5
    API --> Admin : 400 任务饱和
else 可派单
    API -> WO : setMaintainer, status=assigned
    WO -> DB : UPDATE work_order
    API -> DB : INSERT work_order_log
    API --> Admin : 200 派单成功
    note right of M : 登录后可在\n任务列表查看
end
deactivate API
@enduml
```

(四) 工单状态图

工单是系统核心领域对象，其生命周期适合用 UML 状态图描述。状态图与第五章状态机设计一致，涵盖驳回、撤销、返修等异常路径。

主要状态：pending_review(待审核)、pending_dispatch(待派单)、assigned(已派单)、in_progress(处理中)、pending_confirm(待确认)、completed(已完成)、evaluated(已评价)、rejected(已驳回)、cancelled(已撤销)。

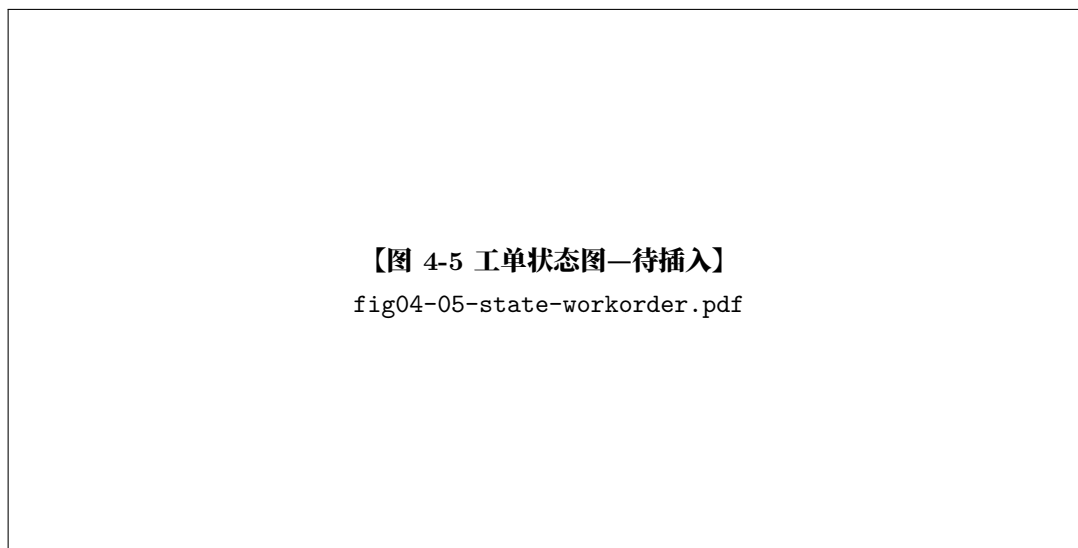


图 13: 报修工单状态图

图 4-5 制作说明（对应作业可选加分项“状态图”）：

1. 类型：UML State Machine Diagram；
2. 初始状态 → 待审核；
3. 迁移标注触发事件：如“提交报修”“审核通过”“派单”“接单”“完工”“学生确认”“提交评价”“驳回”“撤销”“申请返修”；
4. 从待确认：→ 已完成（确认或超时自动确认）；→ 处理中（返修）；
5. 终态：已评价、已驳回、已撤销（用带圈终止符号）；
6. 工具：StarUML Statechart 或 PlantUML state。

PlantUML 参考：

```
@startuml
[*] --> 待审核 : 提交报修
待审核 --> 待派单 : 审核通过
待审核 --> 已驳回 : 驳回
待审核 --> 已撤销 : 撤销
待派单 --> 已派单 : 派单
已派单 --> 处理中 : 接单
处理中 --> 待确认 : 完工
待确认 --> 已完成 : 确认/自动确认
```

```

待确认 --> 处理中 : 返修
已完成 --> 已评价 : 评价
已驳回 --> [*]
已撤销 --> [*]
已评价 --> [*]
@enduml

```

(五) 部署图

部署图描述系统在物理/逻辑节点上的分布，与第五节文字说明对应。课程演示环境可采用简化拓扑；生产环境采用 Nginx + Gunicorn + MySQL。



图 14: 系统部署图

图 4-6 制作说明：

1. **节点**: <<device>> 客户端设备 (PC/手机 + Browser); <<device>> 应用服务器 (Nginx + Gunicorn + Django App); <<device>> 数据库服务器 (MySQL; 开发环境可标 SQLite 文件);
2. **构件**: 在应用服务器节点内嵌套 <<artifact>> dormfix.wsgi;
3. **通信**: HTTP/HTTPS 连接 Browser 与 Nginx; WSGI 连接 Nginx 与 Django; JDBC/ORM 连接 Django 与 DB;
4. **媒体文件**: 旁注 MEDIA_ROOT 由 Nginx 或 Django 提供;
5. 可与第 2 章图 2-4 使用同一图源，保持全文一致;
6. 工具: StarUML Deployment、draw.io 或 PlantUML deployment。

PlantUML 参考：

```

@startuml
node "客户端设备" {
    [Web Browser]
}
node "应用服务器" {

```



```

node "Nginx" as nginx
node "Gunicorn + Django" as app {
    [dormfix App]
}
}
node "数据库服务器" {
    database "MySQL / SQLite" as db
}
[Web Browser] --> nginx : HTTP(S)
nginx --> app : WSGI
app --> db : ORM
@enduml

```

(六) 本章小结

本章给出了 DormFix 的用例图、核心业务类图、提交报修与派发任务两条顺序图、工单状态图及部署图的设计说明与制图规范。第五章将进一步从数据持久化角度给出 E-R 图与数据表结构。

五、 数据库设计

本章说明 DormFix 系统的数据模型、E-R 结构、主要数据表及工单状态机设计。物理实现采用 Django ORM；开发环境使用 SQLite，部署环境可切换为 MySQL，表名与字段名与代码中 `db_table` 及模型定义一致。

(一) 数据库设计原则

本系统数据库设计遵循以下原则：

1. **数据规范化**：实体按业务域拆分（用户与宿舍、工单、评价投诉），避免在工单表中冗余存储楼栋名称等可推导信息；图片、日志独立成表，满足第三范式基本要求；
2. **数据一致性**：通过外键约束与 `on_delete` 策略保证引用完整：学生、房间等关键引用使用 `PROTECT` 防止误删；工单删除时级联删除图片与日志；维修人员解绑使用 `SET_NULL`；
3. **完整性约束**：唯一约束（工单编号、登录账号、楼栋编号）、联合唯一（楼栋 + 房间号）、非空与枚举取值由 Django 模型与数据库共同保证；
4. **可查询性**：对高频筛选字段建立索引，如工单 `status`、`category`，用户 `student_or_staff_no`，房间 `room_no`；
5. **可扩展性**：采用逻辑表名与 Django Migration 管理 `schema` 变更（如报修图片由单字段拆分为 `work_order_image` 表），便于版本迭代与环境迁移。

(二) E-R 图

1. 实体与联系

系统共识别 8 个核心业务实体：用户、宿舍楼、宿舍房间、报修工单、工单图片、工单日志、服务评价、投诉记录。实体间主要联系如下：

- 宿舍楼 (1) ——(N) 宿舍房间;
- 用户 (学生) (1) ——(N) 报修工单;
- 用户 (维修人员) (0..1) ——(N) 报修工单 (派单后关联);
- 宿舍房间 (1) ——(N) 报修工单;
- 报修工单 (1) ——(N) 工单图片;
- 报修工单 (1) ——(N) 工单日志; 用户 (1) ——(N) 工单日志 (操作人);
- 报修工单 (1) ——(0..1) 服务评价;
- 报修工单 (1) ——(N) 投诉记录; 用户 (学生) (1) ——(N) 投诉记录。

【图 5-1 系统 E-R 图—待插入】

fig05-01-er.pdf

图 15: 系统实体联系图 (E-R 图)

图 5-1 制作说明:

1. **表示法**: 采用 Chen 记法或陈氏简化 E-R (矩形 = 实体, 菱形 = 联系, 椭圆 = 属性可选); 若用 Crow's Foot, 须在图注中说明;
2. **实体矩形**内写中文名: 用户、宿舍楼、宿舍房间、报修工单、工单图片、工单日志、服务评价、投诉记录;
3. **联系标注**: 如 “包含”“提交”“指派”“记录”“评价”“投诉”, 并标 1 : N、1 : 1;
4. **主键属性**: 可在实体旁用下划线标出 id、order_no 等, 不必列出全部属性 (详细字段见表16-23);
5. **工具**: draw.io、PowerDesigner、Visio 或 PlantUML ER 风格;
6. **导出**: PDF 矢量, 横向, width=0.92。

draw.io 绘制要点: 以 “报修工单” 为中心实体, 向左连 “用户 (学生)”、“宿舍房间”→“宿舍楼”, 向右连 “工单图片”“工单日志”, 向下连 “服务评价” (1:1)、“投诉记录” (1:N)。

(三) 主要数据表设计

以下各表字段类型按 Django ORM 映射说明: CharField → VARCHAR, TextField → TEXT, DateTimeField → DATETIME, ImageField → 文件路径字符串; 主键均为自增 BIGINT (id)。

1. 用户表 (user)

用户表存储学生、管理员、维修人员账户信息, 继承 Django AbstractUser 部分系统字段; 业务查询以 account 为登录标识。

表 16: 用户表 user 结构

字段名	数据类型	主键	外键	说明
id	BIGINT	是	—	自增主键
account	VARCHAR(50)	—	—	登录账号, 唯一
password	VARCHAR(128)	—	—	密码哈希 (PBKDF2)
name	VARCHAR(50)	—	—	姓名
student_or_staff_no	VARCHAR(30)	—	—	学号/工号, 建索引
phone	VARCHAR(20)	—	—	联系电话
role	VARCHAR(20)	—	—	角色: student/admin/maintainer
status	SMALLINT	—	—	账号状态, 1= 启用, 0= 禁用
login_attempts	INT	—	—	连续登录失败次数, 默认 0
lockout_until	DATETIME	—	—	账户锁定截止时间, 可空
created_at	DATETIME	—	—	创建时间
last_login	DATETIME	—	—	最后登录时间, 可空
is_active	BOOLEAN	—	—	Django 账户是否激活

2. 宿舍楼表 (dorm_building)

表 17: 宿舍楼表 dorm_building 结构

字段名	数据类型	主键	外键	说明
id	BIGINT	是	—	自增主键
building_code	VARCHAR(20)	—	—	楼栋编号, 唯一
building_name	VARCHAR(50)	—	—	楼栋名称
gender_limit	VARCHAR(10)	—	—	性别限制, 可空

3. 宿舍房间表 (dorm_room)

表 18: 宿舍房间表 dorm_room 结构

字段名	数据类型	主键	外键	说明
id	BIGINT	是	—	自增主键
building_id	BIGINT	—	dorm_building.id	所属楼栋, 级联删除
room_no	VARCHAR(20)	—	—	房间号, 与楼栋联合唯一
floor_no	INT	—	—	楼层

约束: (building_id, room_no) 联合唯一。

4. 报修工单表 (work_order)

工单表为系统核心表, 记录报修全生命周期及关联人员、地点。

表 19: 报修工单表 work_order 结构

字段名	数据类型	主键	外键	说明
id	BIGINT	是	—	自增主键
order_no	VARCHAR(40)	—	—	工单编号, 唯一; 格式 WX+ 时间戳 + 随机数
student_id	BIGINT	—	user.id	报修学生, PROTECT
maintainer_id	BIGINT	—	user.id	维修人员, 可空, SET NULL
room_id	BIGINT	—	dorm_room.id	报修房间, PROTECT
category	VARCHAR(30)	—	—	类别: 水电/门窗/网络/家具/其他, 索引
description	TEXT	—	—	故障描述, 最长 1000 字
urgency_level	VARCHAR(10)	—	—	紧急程度: normal/urgent
status	VARCHAR(20)	—	—	工单状态, 见四, 索引
submit_time	DATETIME	—	—	提交时间
assign_time	DATETIME	—	—	派单时间, 可空
finish_time	DATETIME	—	—	完工时间, 可空
cancel_flag	SMALLINT	—	—	是否撤销标记, 默认 0

5. 工单图片表 (work_order_image)

表 20: 工单图片表 work_order_image 结构

字段名	数据类型	主键	外键	说明
id	BIGINT	是	—	自增主键
work_order_id	BIGINT	—	work_order.id	所属工单, 级联删除
image	VARCHAR(100)	—	—	图片存储路径, 上传目录 repair_images/

业务规则: 单工单最多 3 张图片, 单张不超过 5 MB (应用层校验)。

6. 工单日志表 (work_order_log)

表 21: 工单日志表 work_order_log 结构

字段名	数据类型	主键	外键	说明
id	BIGINT	是	—	自增主键
work_order_id	BIGINT	—	work_order.id	所属工单, 级联删除
operator_id	BIGINT	—	user.id	操作人, PROTECT
from_status	VARCHAR(20)	—	—	原状态, 可空
to_status	VARCHAR(20)	—	—	新状态
operation_type	VARCHAR(30)	—	—	操作类型, 如提交报修、审核通过、派单
operation_time	DATETIME	—	—	操作时间
remark	TEXT	—	—	备注, 如驳回理由、维修结果

7. 服务评价表 (evaluation)

表 22: 服务评价表 evaluation 结构

字段名	数据类型	主键	外键	说明
id	BIGINT	是	—	自增主键
work_order_id	BIGINT	—	work_order.id	对应工单, 一对一, 级联删除
speed_score	INT	—	—	维修速度评分, 1-5
attitude_score	INT	—	—	服务态度评分, 1-5
quality_score	INT	—	—	维修质量评分, 1-5
content	TEXT	—	—	文字评价, 可空, 最长 500 字
created_at	DATETIME	—	—	评价时间

8. 投诉记录表 (complaint)

表 23: 投诉记录表 complaint 结构

字段名	数据类型	主键	外键	说明
id	BIGINT	是	—	自增主键
work_order_id	BIGINT	—	work_order.id	关联工单, 级联删除
student_id	BIGINT	—	user.id	投诉人, PROTECT
type	VARCHAR(20)	—	—	类型: quality/attitude/other
content	TEXT	—	—	投诉内容, 最长 1000 字
status	VARCHAR(20)	—	—	处理状态: pending/processing/resolved
process_result	TEXT	—	—	处理结果, 可空
created_at	DATETIME	—	—	创建时间
handled_at	DATETIME	—	—	处理时间, 可空

9. 辅助与系统表说明

除上述业务表外, 运行时尚依赖 Django 框架表, 如 authtoken_token (DRF Token)、django_session 等, 由框架自动维护, 本文不展开列出。

(四) 工单状态设计

工单状态存储于 work_order.status 字段, 采用**有限状态机**管理。状态取值与业务含义见表24。

表 24: 工单状态枚举

状态码	状态名称	说明
pending_review	待审核	学生提交后的初始状态
pending_dispatch	待派单	管理员审核通过
assigned	已派单	已指定维修人员
in_progress	处理中	维修人员已接单
pending_confirm	已完成待确认	维修人员已提交完工
completed	已完成	学生确认或超时自动确认
evaluated	已评价	学生提交服务评价后
rejected	已驳回	管理员审核驳回
cancelled	已撤销	学生在待审核阶段撤销

1. 状态流转规则

主路径（正常报修闭环）：

待审核 → 待派单 → 已派单 → 处理中 → 待确认 → 已完成 → 已评价

对应触发事件分别为：审核通过、派单、接单、完工提交、学生确认（或超时自动确认）、提交评价。

分支与异常路径：

- 待审核 $\xrightarrow{\text{驳回}}$ 已驳回（终态）；
- 待审核 $\xrightarrow{\text{撤销}}$ 已撤销（终态）；
- 待确认 $\xrightarrow{\text{返修}}$ 处理中（学生不满意，填写返修原因）；
- 待确认 $\xrightarrow{\text{满 3 天未操作}}$ 已完成（系统在确认接口中自动迁移，日志记为“系统自动确认”）。

UML 状态图见图13；每次合法迁移均在 `work_order_log` 中记录 `from_status`、`to_status` 与 `operation_type`，保证前台时间线可追溯。

2. 状态—操作权限矩阵（设计约束）

表25 汇总各状态下允许的关键操作，供实现与测试对照。

表 25: 工单状态与允许操作

当前状态	允许操作	执行角色
<code>pending_review</code>	审核通过/驳回、撤销	管理员/学生
<code>pending_dispatch</code>	派单	管理员
<code>assigned</code>	接单	维修人员
<code>in_progress</code>	完工提交	维修人员
<code>pending_confirm</code>	确认/返修、自动确认	学生/系统
<code>completed</code>	提交评价、投诉	学生
<code>evaluated</code>	投诉（只读查看为主）	学生/管理员
<code>rejected, cancelled</code>	无后续流转	—

（五） 本章小结

本章阐述了数据库设计原则与 8 个核心业务实体的 E-R 关系，给出了各主要数据表的字段、类型、键约束及说明，并定义了工单状态枚举、流转规则与状态—操作约束。第六章将从安全性、性能等方面说明系统质量设计。

六、 系统质量设计

本章从安全性、性能、异常处理、日志审计与可扩展性等方面说明 DormFix 的质量属性设计，并给出与需求分析文档非功能性需求的对应关系。所述**已实现**措施均可在当前代码库中验证；**规划**措施用于生产部署与后续迭代。

（一） 安全性设计

1. 身份认证与访问控制

- Token 认证（已实现）**：采用 DRF `TokenAuthentication`，用户登录成功后颁发 Token，前端将 Token 存入 `sessionStorage`，后续请求经 `api.js` 统一在请求头附加 `Authorization: Token <token>`；登出时删除服务端 Token 记录。

- 2. **默认鉴权（已实现）**: REST_FRAMEWORK 配置默认权限为 `IsAuthenticated`, 除登录接口外业务 API 均需认证; 未携带或 Token 无效时返回 HTTP 401, 前端自动跳转登录页。
- 3. **基于角色的权限控制（已实现）**: 各视图函数内按 `user.is_student`、`is_admin`、`is_maintainer` 判断操作合法性; 例如仅学生可创建报修, 仅管理员可审核派单, 仅指派维修人员可接单完工; 学生查询工单时校验 `order.student == request.user`, 实现数据隔离。
- 4. **账户锁定（已实现）**: 连续登录失败 5 次锁定 15 分钟, 由 `login_attempts`、`lockout_until` 字段及 `login_view` 逻辑实现; 禁用账户 (`status=0`) 禁止登录。

2. 密码与数据安全

- **密码存储（已实现）**: 使用 Django 内置 PBKDF2 算法哈希存储, 满足“禁止明文密码”要求; 重置密码由管理员接口调用 `set_password()` 完成;
- **传输安全（规划）**: 课程演示环境使用 HTTP 本地访问; 生产部署应在 Nginx 层配置 TLS 证书, 启用 HTTPS, 满足需求分析中的加密传输要求;
- **敏感配置（规划）**: `SECRET_KEY`、数据库口令等应通过环境变量注入, 禁止将生产密钥写入版本库;
- **输入校验（已实现）**: DRF 序列化器校验登录、用户创建等字段; 业务视图对必填项、枚举取值、图片数量与大小进行校验;
- **评价内容过滤（已实现）**: `feedback` 模块维护敏感词列表, 命中则拒绝提交评价。

3. Web 安全防护

- **CSRF（部分适用）**: Django 中间件启用 `CsrfViewMiddleware`; JSON API 主要依赖 Token 认证, 表单页面在需要时可配合 CSRF Token;
- **点击劫持**: `XFrameOptionsMiddleware` 限制页面被嵌套;
- **CORS**: 开发阶段配置 `CORS_ALLOW_ALL_ORIGINS=True` 便于调试; 生产环境应收紧为指定域名白名单;
- **XSS 缓解（已实现/持续改进）**: 列表与详情展示优先使用 `textContent` 渲染用户文本; 对动态 HTML 拼接处应在后续版本中统一封装转义函数, 降低脚本注入风险。

表 26: 安全性需求与设计落实对照

需求要点	状态	设计/实现说明
账号密码认证	已实现	Token + 账号/学号登录
角色权限控制	已实现	视图层 RBAC + 数据隔离
未登录禁止访问	已实现	DRF 默认 <code>IsAuthenticated</code>
登录失败锁定	已实现	5 次 / 15 分钟
密码哈希存储	已实现	Django PBKDF2
HTTPS 传输	规划	Nginx TLS 终止
关键操作审计	已实现	<code>work_order_log</code> 表
数据库备份恢复	规划	运维侧定时备份策略

(二) 性能优化设计

1. 数据库层优化（已实现）

- 对工单 status、category 建立数据库索引，加速按状态、类别筛选的列表查询；
- 对用户 student_or_staff_no、房间 room_no 建索引，支持学号/工号登录及房间检索；
- 外键关联查询在序列化器中适度使用，避免 N+1 问题可在后续通过 select_related / prefetch_related 进一步优化。

2. 应用层与接口层优化（已实现）

- **分页**：列表接口统一采用 PageNumberPagination，默认每页 10 条，前端通过 page、page_size 参数请求，降低单次响应数据量；
- **图片约束**：报修图片限制数量与单文件大小（3 张 / 5 MB），控制存储与传输开销；
- **请求超时**：前端 api.js 设置 15s 超时（AbortController），避免长时间挂起；
- **统计查询**：dashboard 模块使用 ORM 聚合（Count、Avg 等）一次计算指标，减少往返次数。

3. 缓存与静态资源（部分实现 / 规划）

- **开发环境**：DisableCacheMiddleware 禁用页面缓存，便于调试；
- **前端会话缓存**：sessionStorage 缓存 Token 与用户摘要信息，减少重复请求个人信息；
- **服务端缓存（规划）**：生产环境可引入 Redis 缓存热点统计结果、宿舍楼房间字典等读多写少数据；
- **静态资源（规划）**：Nginx 直接托管 static/、media/，并开启 gzip 与浏览器缓存策略。

表 27: 性能优化措施一览

优化项	状态	说明
列表分页	已实现	默认 10 条/页
字段索引	已实现	状态、类别、学号等
图片大小限制	已实现	应用层校验
接口超时控制	已实现	前端 15s
聚合统计	已实现	Dashboard ORM 聚合
Redis 缓存	规划	热点数据、会话可选
CDN / 静态加速	规划	生产静态资源

(三) 异常处理机制

系统采用“分层校验 + 统一 HTTP 语义 + 前端友好提示”的异常处理策略。

1. 后端异常处理（已实现）

1. **参数校验**：DRF 序列化器在登录、用户创建等场景执行字段级校验，失败返回 HTTP 400 及字段错误字典；

2. **业务规则校验**: 各视图对非法状态迁移、权限不足、业务上限 (未完成工单数、日派单数) 返回 400/403 及 error 消息;
3. **资源不存在**: 工单、用户不存在时返回 HTTP 404;
4. **认证失败**: Token 无效或缺失由 DRF 返回 401;
5. **事务原子性**: 工单创建、状态变更、日志写入在同一请求处理流程中完成, 单条请求内由 Django ORM 保证写入一致性; 跨表复杂场景可在后续引入 `transaction.atomic()` 显式包裹。

2. 前端异常处理 (已实现)

`api.js` 对非 2xx 响应解析 JSON 错误体并 `throw`, 各页面 `catch` 后以 Toast 或表单旁注展示; 网络超时、401 认证过期分别提示 “请求超时” 与跳转登录页, 避免未处理异常导致页面无响应。

3. 典型异常场景

表 28: 典型异常场景与处理

场景	HTTP	处理方式
未完成工单 ≥ 3 仍报修	403	返回中文错误, 前端阻止提交
非待审核状态撤销	400	提示无法撤销
派单日任务饱和	400	提示更换维修人员
非指派人员接单	404/400	工单不存在或状态不允许
重复评价	400	提示已评价
Token 过期	401	清除本地 Token 并跳转登录
请求超时	—	前端 <code>AbortError</code> 提示

(四) 日志与监控设计

1. 业务审计日志 (已实现)

系统以 `work_order_log` 表作为**核心业务审计日志**, 记录工单状态每一次合法变更, 字段包括操作人、原/新状态、操作类型、时间戳与备注。覆盖场景包括: 提交报修、审核通过/驳回、派单、接单、完工、确认/返修、自动确认、评价等。前台工单详情页以时间线形式展示, 满足 “关键业务可追溯” 要求。

2. 系统运行日志 (部分实现 / 规划)

- **已实现**: Django 默认日志框架可在 `settings` 中配置按模块输出 INFO/ERROR 至文件; 开发调试阶段视图异常会输出至控制台;
- **规划**: 生产环境将日志按日期滚动写入文件, 记录时间、请求路径、用户 ID、异常堆栈; 敏感字段 (密码、Token) 禁止写入日志明文;
- **规划**: 对接监控平台 (如 Prometheus + Grafana 或云监控), 采集 QPS、错误率、响应时间等指标, 设置告警阈值。

(五) 可扩展性设计

系统在架构与数据模型上预留以下扩展空间, 便于持续演进。

1. 功能扩展

- **微信小程序 / 移动端**: 后端已提供 REST API, 可在保持 Token 认证前提下新增小程序前端, 复用现有接口;
- **自动派单**: 在 dispatch 模块增加派单策略引擎 (按类别、负载、紧急程度自动选择维修人员), 人工派单作为兜底;
- **消息推送**: 在工单状态变更时触发短信/邮件/企业微信通知, 可通过 Django Signal 或异步任务队列 (Celery) 解耦;
- **智能故障分类**: 基于历史工单文本训练分类模型, 辅助学生选择报修类别;
- **报修类别配置化**: 将 CATEGORY_CHOICES 迁移至字典表, 管理员可在后台维护, 无需修改代码;
- **角色扩展**: 在 role 字段或独立权限表中增加楼栋管理员等角色, 配合细粒度权限类 (DRF Permission) 实现。

2. 技术扩展

- **数据库切换**: ORM 抽象使 SQLite 与 MySQL 切换仅需修改 DATABASES 配置;
- **前后端完全分离**: 现有 API 已覆盖主流程, 可逐步将 Template 页面替换为 Vue/React SPA, 后端仅保留 API;
- **读写分离与分库**: 工单量显著增长时, 可将统计查询路由至只读副本;
- **文件存储**: 报修图片可迁移至对象存储 (OSS/S3), 数据库仅保存 URL。

表 29: 可扩展性规划对照

扩展方向	优先级	实现思路
微信小程序	高	复用 REST API + 新前端
消息通知	高	Signal / 消息队列
自动派单	中	派单策略服务
类别配置化	中	字典表 + 管理界面
智能分类	低	ML 模型 + 推理接口
对象存储	中	替换本地 MEDIA_ROOT

(六) 本章小结

本章从安全、性能、异常、日志与扩展五个维度阐述了 DormFix 的质量设计, 并结合需求分析给出了落实状态与规划项。系统已在认证授权、业务审计、分页索引与异常反馈等方面形成可验证实现; HTTPS、Redis、集中监控与自动备份等将在生产部署阶段完善。第七章将对全文设计及实现成果进行总结。

七、 总结

本章对 DormFix 宿舍报修与维修进度管理系统的软件设计工作进行归纳, 概括设计成果与架构特点, 说明与代码实现的一致性, 并指出后续优化方向。

（一） 系统设计成果

围绕课程《软件设计报告》要求，项目组在需求分析文档基础上完成了面向实现的系统化设计，主要成果包括：

1. **总体设计**：明确了 B/S 架构、分层结构及六个 Django 应用模块的职责划分，给出了功能结构、逻辑架构、模块结构、技术选型与部署方案；
2. **详细设计**：完成了用户管理、报修、派单、维修、评价投诉、数据统计等模块的功能与业务规则说明，绘制核心业务流程，并整理了 REST 接口规范；
3. **UML 建模**：编制用例图、类图、顺序图（提交报修、派发任务）、工单状态图及部署图，与需求分析用例形成可追溯对应；
4. **数据库设计**：建立 8 个核心业务实体的 E-R 模型与数据表结构，定义工单的九种状态及状态—操作约束；
5. **质量设计**：从安全、性能、异常处理、日志审计、可扩展性等方面落实非功能性需求，并区分已实现措施与生产环境规划项；
6. **系统实现**：设计内容与仓库代码一致，系统可在本地环境完成登录、报修、审核派单、维修确认、评价投诉及统计导出等端到端演示。

（二） 架构与设计特点

本系统在架构与设计方法上具有以下特点：

- **分层清晰、职责单一**：用户层、表示层、API 层、业务层、数据层边界明确，利于团队分工与单元测试；
- **领域模块内聚**：以工单为中心，repairs 承载核心实体，dispatch、maintenance 复用模型避免重复建表；
- **状态机驱动流程**：工单全生命周期由有限状态机约束，配合 work_order_log 实现可追溯时间线；
- **半分离前后端**：Django Template 保障页面交付效率，DRF 提供标准 API，便于联调、测试与后续替换前端；
- **设计—实现一致**：类图、数据表、接口路径与 ORM、URL 配置相互印证，降低“空泛设计”风险。

（三） 实际实现情况

系统当前已在课程项目环境中完成主要功能开发与联调，实现情况与设计文档对应关系如下。

表 30: 设计项与实现对照

设计内容	实现状态	说明
三类角色与 Token 认证	已实现	accounts 登录/登出，sessionStorage 存 Token
报修与图片上传	已实现	最多 3 张、5 MB 限制，工单编号自动生成
审核、派单、负载规则	已实现	日派单上限 5，驳回须填理由
接单、完工、确认/返修	已实现	含 3 日自动确认逻辑
评价与投诉	已实现	三维评分、敏感词过滤、投诉处理
统计与 Excel 导出	已实现	ECharts 图表 + openpyxl
工单操作日志	已实现	work_order_log + 详情时间线
HTTPS / Redis 缓存	未部署	见第五节规划

与需求分析文档相比，本期实现采用**管理员人工派单**，未实现需求中设想的自动派单与消息提醒；该差异已在第五章作为扩展项说明。系统在功能范围上仍覆盖“报修—审核—派单—维修—确认—评价/投诉—统计”完整业务闭环，达到预期目标。

(四) 后续优化方向

结合第五章质量与扩展设计，后续工作可优先从以下方面推进：

1. **生产化部署**：配置 Nginx + Gunicorn + MySQL，启用 HTTPS，将密钥与数据库配置环境变量化，制定备份与恢复流程；
2. **体验增强**：增加工单状态变更的短信/微信通知，减少学生反复查询；
3. **智能与自动化**：研究按故障类别、维修人员负载的自动派单策略；
4. **运维可观测性**：完善应用日志、错误监控与性能指标采集；
5. **安全加固**：收紧 CORS、统一输出转义、引入系统级操作审计表；
6. **多端接入**：基于现有 API 开发微信小程序，扩大报修入口覆盖面。

(五) 结语

综上所述，DormFix 项目从需求分析到软件设计、编码实现形成了较完整的软件工程实践链条。本报告在架构、模块、流程、接口、UML、数据库及质量属性等方面给出了可评审、可验证的设计说明，为系统演示、课程验收及后续迭代奠定了基础。